

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python



File Edit View Run Help

Last edit was 5 minutes ago

Provide feedback

Run all

Connect

Schedule

Share

Cmd 1

```
1 %run ./01-config
```

Cmd 2

```
1 class Upsserter:
2     def __init__(self, merge_query, temp_view_name):
3         self.merge_query = merge_query
4         self.temp_view_name = temp_view_name
5
6     def upsert(self, df_micro_batch, batch_id):
7         df_micro_batch.createOrReplaceTempView(self.temp_view_name)
8         df_micro_batch._jdf.sparkSession().sql(self.merge_query)
```

Cmd 3

1

Python

Shift+Enter to run

Shift+Ctrl+Enter to run selected text

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python



File Edit View Run Help Last edit was 5 minutes ago Provide feedback

Run all

Connect

Schedule

Share

```
7 df_micro_batch.createOrReplaceTempView(self.temp_view_name)
8 df_micro_batch._jdf.sparkSession().sql(self.merge_query)
```

Cmd 3

```
1 class CDCUpsserter:
2     def __init__(self, merge_query, temp_view_name, id_column, sort_by):
3         self.merge_query = merge_query
4         self.temp_view_name = temp_view_name
5         self.id_column = id_column
6         self.sort_by = sort_by
7
8     def upsert(self, df_micro_batch, batch_id):
9         from pyspark.sql.window import Window
10        from pyspark.sql import functions as F
11
12        window = Window.partitionBy(self.id_column).orderBy(F.col(self.sort_by).desc())
13
14        df_micro_batch.filter(F.col("update_type").isin(["new", "update"])) \
15            .withColumn("rank", F.rank().over(window)).filter("rank == 1").drop("rank") \
16            .createOrReplaceTempView(self.temp_view_name)
17        df_micro_batch._jdf.sparkSession().sql(self.merge_query)
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text

+ New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python ▾

☆

File Edit View Run Help

[Last edit was 5 minutes ago](#)[Provide feedback](#)

▶ Run all

● Connect ▾

📅 Schedule

Share



7

df_micro_batch.createOrReplaceTempView(self.temp_view_name)

8

df_micro_batch._jdf.sparkSession().sql(self.merge_query)



Cmd 3

```
1 class CDCUpserter:
2     def __init__(self, merge_query, temp_view_name, id_column, sort_by):
3         self.merge_query = merge_query
4         self.temp_view_name = temp_view_name
5         self.id_column = id_column
6         self.sort_by = sort_by
7
8     def upsert(self, df_micro_batch, batch_id):
9         from pyspark.sql.window import Window
10        from pyspark.sql import functions as F
11
12        window = Window.partitionBy(self.id_column).orderBy(F.col(self.sort_by).desc())
13
14        df_micro_batch.filter(F.col("update_type").isin(["new", "update"])) \
15            .withColumn("rank", F.rank().over(window)).filter("rank == 1").drop("rank") \
16            .createOrReplaceTempView(self.temp_view_name)
17        df_micro_batch._jdf.sparkSession().sql(self.merge_query)
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver initial-feature Python

File Edit View Run Help Last edit was 6 minutes ago Provide feedback

Run all

Connect

Schedule

Share

17 dt_micro_batch._jvm.sparkSession().sql(self.merge_query)

Cmd 4

```
1 class Silver():
2     def __init__(self, env):
3         self.Conf = Config()
4         self.checkpoint_base = self.Conf.base_dir_checkpoint + "/checkpoints"
5         self.catalog = env
6         self.db_name = self.Conf.db_name
7         self.maxFilesPerTrigger = self.Conf.maxFilesPerTrigger
8         spark.sql(f"USE {self.catalog}.{self.db_name}")
9
10    def upsert_users(self, once=True, processing_time="15 seconds", startingVersion=0):
11        from pyspark.sql import functions as F
12
13        #Idempotent - User cannot register again so ignore the duplicates and insert the new records
14        query = f"""
15            MERGE INTO {self.catalog}.{self.db_name}.users a
16            USING users_delta b
17            ON a.user_id=b.user_id
18            WHEN NOT MATCHED THEN INSERT *
19            """
20
21        data_upserter=Upsserter(query, "users_delta")
22
23        # Spark Structured Streaming accepts append only sources
```

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python



File Edit View Run Help

Last edit was 7 minutes ago

Provide feedback

Run all

Connect

Schedule

Share

```
24 # - This is not a problem for silver layer streams because bronze layer is insert only
25 # - However, you may want to allow bronze layer deletes due to regulatory compliance
26 # Spark Structured Streaming throws an exception if any modifications occur on the table being used as a source
27 # - This is a problem for silver layer streaming jobs.
28 # - ignoreDeletes allows to delete records on partition column in the bronze layer without exception on
   silver layer streams
29 # Starting version is to allow you to restart your stream from a given version just in case you need it
30 # - startingVersion is only applied for an empty checkpoint
31 # Limiting your input stream size is critical for running on limited capacity
32 # - maxFilesPerTrigger/maxBytesPerTrigger can be used with the readStream
33 # - Default value is 1000 for maxFilesPerTrigger and maxBytesPerTrigger has no default value
34 # - The recommended maxFilesPerTrigger is equal to #executors assuming auto optimize file size to 128 MB
35 df_delta = (spark.readStream
36             .option("startingVersion", startingVersion)
37             .option("ignoreDeletes", True)
38             #.option("withEventTimeOrder", "true")
39             #.option("maxFilesPerTrigger", self.maxFilesPerTrigger)
40             .table(f"{self.catalog}.{self.db_name}.registered_users_bz")
41             .selectExpr("user_id", "device_id", "mac_address", "cast(registration_timestamp as timestamp)")
42             .withWatermark("registration_timestamp", "30 seconds")
43             .dropDuplicates(["user_id", "device_id"])
44             )
45
46 # We read new records in bronze layer and insert them to silver. So, the silver layer is insert only in a
   typical case.
47 # However, we want to ignoreDeletes, remove duplicates and also merge with an update statement depending upon
   the scenarios
```

+ New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python

☆

File Edit View Run Help

Last edit was 7 minutes ago

Provide feedback

▶ Run all

● Connect

📅 Schedule

Share

```
48 # Hence, it is recommended to set the update mode
49 stream_writer = (df_delta.writeStream
50                  .foreachBatch(data_upserter.upsert)
51                  .outputMode("update")
52                  .option("checkpointLocation", f"{self.checkpoint_base}/users")
53                  .queryName("users_upsert_stream")
54                  )
55
56 spark.sparkContext.setLocalProperty("spark.scheduler.pool", "silver_p2")
57
58 if once == True:
59     return stream_writer.trigger(availableNow=True).start()
60 else:
61     return stream_writer.trigger(processingTime=processing_time).start()
62
63
64 def upsert_gym_logs(self, once=True, processing_time="15 seconds", startingVersion=0):
65     from pyspark.sql import functions as F
66
67     # Idempotent - Insert new login records
68     # - Update logout time when
69     #     1. It is greater than login time
70     #     2. It is greater than earlier logout
71     #     3. It is not NULL (This is also satisfied by above conditions)
72     query = f"""
73     MERGE INTO {self.catalog}.{self.db_name}.gym_logs a
74     USING gym_logs_delta b
```


+ New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python ▾

☆

File Edit View Run Help

Last edit was 17 minutes ago

Provide feedback

▶ Run all

● Connect ▾

📅 Schedule

Share

```
74 USING gym_logs_delta b
75 ON a.mac_address=b.mac_address AND a.gym=b.gym AND a.login=b.login
76 WHEN MATCHED AND b.logout > a.login AND b.logout > a.logout
77 THEN UPDATE SET logout = b.logout
78 WHEN NOT MATCHED THEN INSERT *
79
80
81 data_upserter=Upsserter(query, "gym_logs_delta")
82
83 df_delta = (spark.readStream
84             .option("startingVersion", startingVersion)
85             .option("ignoreDeletes", True)
86             #.option("withEventTimeOrder", "true")
87             #.option("maxFilesPerTrigger", self.maxFilesPerTrigger)
88             .table(f"{self.catalog}.{self.db_name}.gym_logins_bz")
89             .selectExpr("mac_address", "gym", "cast(login as timestamp)", "cast(logout as timestamp)")
90             .withWatermark("login", "30 seconds")
91             .dropDuplicates(["mac_address", "gym", "login"])
92             )
93
94 stream_writer = (df_delta.writeStream
95                  .foreachBatch(data_upserter.upsert)
96                  .outputMode("update")
97                  .option("checkpointLocation", f"{self.checkpoint_base}/gym_logs")
98                  .queryName("gym_logs_upsert_stream")
99                  )
100
```

+ New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python



File Edit View Run Help

Last edit was 17 minutes ago

Provide feedback

▶ Run all

● Connect ▾

📅 Schedule

Share

```
81 data_upserter=Upsserter(query, "gym_logs_delta")
82
83 df_delta = (spark.readStream
84             .option("startingVersion", startingVersion)
85             .option("ignoreDeletes", True)
86             #.option("withEventTimeOrder", "true")
87             #.option("maxFilesPerTrigger", self.maxFilesPerTrigger)
88             .table(f"{self.catalog}.{self.db_name}.gym_logins_bz")
89             .selectExpr("mac_address", "gym", "cast(login as timestamp)", "cast(logout as timestamp)")
90             .withWatermark("login", "30 seconds")
91             .dropDuplicates(["mac_address", "gym", "login"])
92             )
93
94 stream_writer = (df_delta.writeStream
95                 .foreachBatch(data_upserter.upsert)
96                 .outputMode("update")
97                 .option("checkpointLocation", f"{self.checkpoint_base}/gym_logs")
98                 .queryName("gym_logs_upsert_stream")
99                 )
100
101 spark.sparkContext.setLocalProperty("spark.scheduler.pool", "silver_p3")
102
103 if once == True:
104     return stream_writer.trigger(availableNow=True).start()
105 else:
106     return stream_writer.trigger(processingTime=processing_time).start()
107
```

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python



File Edit View Run Help

Last edit was 17 minutes ago

Provide feedback

Run all

Connect

Schedule

Share

```
107
108
109 def upsert_user_profile(self, once=False, processing_time="15 seconds", startingVersion=0):
110     from pyspark.sql import functions as F
111
112     #Idempotent - Insert new record
113     #             - Ignore deletes
114     #             - Update user details when
115     #                 1. update_type in ("new", "append")
116     #                 2. current update is newer than the earlier
117     schema = """
118         user_id bigint, update_type STRING, timestamp FLOAT,
119         dob STRING, sex STRING, gender STRING, first_name STRING, last_name STRING,
120         address STRUCT<street_address: STRING, city: STRING, state: STRING, zip: INT>
121     """
122
123     query = f"""
124         MERGE INTO {self.catalog}.{self.db_name}.user_profile a
125         USING user_profile_cdc b
126         ON a.user_id=b.user_id
127         WHEN MATCHED AND a.updated < b.updated
128         | THEN UPDATE SET *
129         WHEN NOT MATCHED
130         | THEN INSERT *
131     """
132
133     data_upserter = CDCUpsserter(query, "user_profile_cdc", "user_id", "updated")
```

05-silver - Databricks

adb-3007342644227763.3.azuredatabricks.net/?o=3007342644227763#notebook/3430256633760883/command/3430256633760889

Microsoft Azure databricks Search data, notebooks, recents, and more... CTRL + P sbit-ws prashant@scholarnest.com

NewWorkspaceRecentsDataWorkflowsComputeSQL EditorQueriesDashboardsAlertsQuery HistorySQL WarehousesData EngineeringJob RunsData IngestionDelta Live TablesMachine Learning

05-silverinitial-featurePython☆FileEditViewRunHelpLast edit was 17 minutes agoProvide feedbackRun allConnectScheduleShare

```
135 df_cdc = (spark.readStream
136         .option("startingVersion", startingVersion)
137         .option("ignoreDeletes", True)
138         #.option("withEventTimeOrder", "true")
139         #.option("maxFilesPerTrigger", self.maxFilesPerTrigger)
140         .table(f"{self.catalog}.{self.db_name}.kafka_multiplex_bz")
141         .filter("topic = 'user_info'")
142         .select(F.from_json(F.col("value").cast("string"), schema).alias("v"))
143         .select("v.*")
144         .select("user_id", F.to_date('dob', 'MM/dd/yyyy').alias('dob'),
145               'sex', 'gender', 'first_name', 'last_name', 'address.*',
146               F.col('timestamp').cast("timestamp").alias("updated"),
147               "update_type")
148         .withWatermark("updated", "30 seconds")
149         .dropDuplicates(["user_id", "updated"])
150     )
151
152     stream_writer = (df_cdc.writeStream
153                     .foreachBatch(data_upserter.upsert)
154                     .outputMode("update")
155                     .option("checkpointLocation", f"{self.checkpoint_base}/user_profile")
156                     .queryName("user_profile_stream")
157                     )
158
159     spark.sparkContext.setLocalProperty("spark.scheduler.pool", "silver_p3")
160
```

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python



File Edit View Run Help Last edit was 17 minutes ago Provide feedback

Run all

Connect

Schedule

Share

```
161         if once == True:
162             return stream_writer.trigger(availableNow=True).start()
163         else:
164             return stream_writer.trigger(processingTime=processing_time).start()
165
166
167     def upsert_workouts(self, once=False, processing_time="10 seconds", startingVersion=0):
168         from pyspark.sql import functions as F
169         schema = "user_id INT, workout_id INT, timestamp FLOAT, action STRING, session_id INT"
170
171         #Idempotent - User cannot have two workout sessions at the same time. So ignore the duplicates and insert the
172         #new records
173         query = f"""
174             MERGE INTO {self.catalog}.{self.db_name}.workouts a
175             USING workouts_delta b
176             ON a.user_id=b.user_id AND a.time=b.time
177             WHEN NOT MATCHED THEN INSERT *
178             """
179
180         data_upserter=Upsserter(query, "workouts_delta")
181
182         df_delta = (spark.readStream
183                     .option("startingVersion", startingVersion)
184                     .option("ignoreDeletes", True)
185                     #.option("withEventTimeOrder", "true")
186                     #.option("maxFilesPerTrigger", self.maxFilesPerTrigger)
187                     .table(f"{self.catalog}.{self.db_name}.kafka_multiplex_bz")
```

```
177     """
178
179     data_upserter=Upsserter(query, "workouts_delta")
180
181     df_delta = (spark.readStream
182                 .option("startingVersion", startingVersion)
183                 .option("ignoreDeletes", True)
184                 #.option("withEventTimeOrder", "true")
185                 #.option("maxFilesPerTrigger", self.maxFilesPerTrigger)
186                 .table(f"{self.catalog}.{self.db_name}.kafka_multiplex_bz")
187                 .filter("topic = 'workout'")
188                 .select(F.from_json(F.col("value").cast("string"), schema).alias("v"))
189                 .select("v.*")
190                 .select("user_id", "workout_id",
191                        F.col("timestamp").cast("timestamp").alias("time"),
192                        "action", "session_id")
193                 .withWatermark("time", "30 seconds")
194                 .dropDuplicates(["user_id", "time"])
195                 )
196
197     stream_writer = (df_delta.writeStream
198                     .foreachBatch(data_upserter.upsert)
199                     .outputMode("update")
200                     .option("checkpointLocation", f"{self.checkpoint_base}/workouts")
201                     .queryName("workouts_upsert_stream")
202                     )
```

05-silver - Databricks

adb-3007342644227763.3.azure.databricks.net/?o=3007342644227763#notebook/3430256633760883/command/3430256633760889

Microsoft Azure databricks

Search data, notebooks, recents, and more... CTRL + P

sbit-ws

prashant@scholarnest.com

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python

☆

File

Edit

View

Run

Help

Last edit was 17 minutes ago

Provide feedback

Run all

Connect

Schedule

Share

```
203
204     spark.sparkContext.setLocalProperty("spark.scheduler.pool", "silver_p3")
205
206     if once == True:
207         return stream_writer.trigger(availableNow=True).start()
208     else:
209         return stream_writer.trigger(processingTime=processing_time).start()
210
211
212 def upsert_heart_rate(self, once=False, processing_time="10 seconds", startingVersion=0):
213     from pyspark.sql import functions as F
214
215     schema = "device_id LONG, time TIMESTAMP, heartrate DOUBLE"
216
217     #Idempotent - Only one BPM signal is allowed at a timestamp. So ignore the duplicates and insert the new records
218     query = f"""
219     MERGE INTO {self.catalog}.{self.db_name}.heart_rate a
220     USING heart_rate_delta b
221     ON a.device_id=b.device_id AND a.time=b.time
222     WHEN NOT MATCHED THEN INSERT *
223     """
224
225     data_upserter=Upsserter(query, "heart_rate_delta")
226
227     df_delta = (spark.readStream
228                 .option("startingVersion", startingVersion)
229                 .option("ignoreDeletes", True)
```

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver initial-feature Python

File Edit View Run Help Last edit was 17 minutes ago Provide feedback

Run all

Connect

Schedule

Share

```
227 df_delta = (spark.readStream
228             .option("startingVersion", startingVersion)
229             .option("ignoreDeletes", True)
230             #.option("withEventTimeOrder", "true")
231             #.option("maxFilesPerTrigger", self.maxFilesPerTrigger)
232             .table(f"{self.catalog}.{self.db_name}.kafka_multiplex_bz")
233             .filter("topic = 'bpm'")
234             .select(F.from_json(F.col("value").cast("string"), schema).alias("v"))
235             .select("v.*", F.when(F.col("v.heartrate") <= 0, False).otherwise(True).alias("valid"))
236             .withWatermark("time", "30 seconds")
237             .dropDuplicates(["device_id", "time"])
238             )
239
240 stream_writer = (df_delta.writeStream
241                  .foreachBatch(data_upserter.upsert)
242                  .outputMode("update")
243                  .option("checkpointLocation", f"{self.checkpoint_base}/heart_rate")
244                  .queryName("heart_rate_upsert_stream")
245                  )
246
247 spark.sparkContext.setLocalProperty("spark.scheduler.pool", "silver_p2")
248
249 if once == True:
250     return stream_writer.trigger(availableNow=True).start()
251 else:
252     return stream_writer.trigger(processingTime=processing_time).start()
253
```

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python



File Edit View Run Help Last edit was 18 minutes ago Provide feedback

Run all

Connect

Schedule

Share

```
253
254
255 def age_bins(self, dob_col):
256     from pyspark.sql import functions as F
257     age_col = F.floor(F.months_between(F.current_date(), dob_col)/12).alias("age")
258     return (F.when((age_col < 18), "under 18")
259             .when((age_col >= 18) & (age_col < 25), "18-25")
260             .when((age_col >= 25) & (age_col < 35), "25-35")
261             .when((age_col >= 35) & (age_col < 45), "35-45")
262             .when((age_col >= 45) & (age_col < 55), "45-55")
263             .when((age_col >= 55) & (age_col < 65), "55-65")
264             .when((age_col >= 65) & (age_col < 75), "65-75")
265             .when((age_col >= 75) & (age_col < 85), "75-85")
266             .when((age_col >= 85) & (age_col < 95), "85-95")
267             .when((age_col >= 95), "95+")
268             .otherwise("invalid age").alias("age"))
269
270
271 def upsert_user_bins(self, once=True, processing_time="15 seconds", startingVersion=0):
272     from pyspark.sql import functions as F
273
274     # Idempotent - This table is maintained as SCD Type 1 dimension
275     #             - Insert new user_id records
276     #             - Update old records using the user_id
277
278     query = f"""
```

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver initial-feature Python

File Edit View Run Help Last edit was 18 minutes ago Provide feedback

Run all

Connect

Schedule

Share

```
# Idempotent - This table is maintained as SCD Type 1 dimension
#           - Insert new user_id records
#           - Update old records using the user_id
```

```
query = f"""
```

```
    MERGE INTO {self.catalog}.{self.db_name}.user_bins a
    USING user_bins_delta b
    ON a.user_id=b.user_id
    WHEN MATCHED
    THEN UPDATE SET *
    WHEN NOT MATCHED THEN INSERT *
    """
```

```
data_upserter=Upsserter(query, "user_bins_delta")
```

```
df_user = spark.table(f"{self.catalog}.{self.db_name}.users").select("user_id")
```

```
# Running stream on silver table requires ignoreChanges
# No watermark required - Stream to static join is stateless
df_delta = (spark.readStream
```

```
    .option("startingVersion", startingVersion)
    .option("ignoreChanges", True)
    #.option("withEventTimeOrder", "true")
    #.option("maxFilesPerTrigger", self.maxFilesPerTrigger)
    .table(f"{self.catalog}.{self.db_name}.user_profile")
    .join(df_user, ["user_id"], "left")
```

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python



File Edit View Run Help Last edit was 18 minutes ago Provide feedback

Run all

Connect

Schedule

Share

```
290
291     # Running stream on silver table requires ignoreChanges
292     # No watermark required - Stream to static join is stateless
293     df_delta = (spark.readStream
294                 .option("startingVersion", startingVersion)
295                 .option("ignoreChanges", True)
296                 #.option("withEventTimeOrder", "true")
297                 #.option("maxFilesPerTrigger", self.maxFilesPerTrigger)
298                 .table(f"{self.catalog}.{self.db_name}.user_profile")
299                 .join(df_user, ["user_id"], "left")
300                 .select("user_id", self.age_bins(F.col("dob")), "gender", "city", "state")
301                 )
302
303     stream_writer = (df_delta.writeStream
304                     .foreachBatch(data_upserter.upsert)
305                     .outputMode("update")
306                     .option("checkpointLocation", f"{self.checkpoint_base}/user_bins")
307                     .queryName("user_bins_upsert_stream")
308                     )
309
310     spark.sparkContext.setLocalProperty("spark.scheduler.pool", "silver_p3")
311
312     if once == True:
313         return stream_writer.trigger(availableNow=True).start()
314     else:
315         return stream_writer.trigger(processingTime=processing_time).start()
316
```

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python



File Edit View Run Help Last edit was 18 minutes ago Provide feedback

Run all

Connect

Schedule

Share

```
318
319 def upsert_completed_workouts(self, once=True, processing_time="15 seconds", startingVersion=0):
320     from pyspark.sql import functions as F
321
322     #Idempotent - Only one user workout session completes. So ignore the duplicates and insert the new records
323     query = f"""
324     MERGE INTO {self.catalog}.{self.db_name}.completed_workouts a
325     USING completed_workouts_delta b
326     ON a.user_id=b.user_id AND a.workout_id = b.workout_id AND a.session_id=b.session_id
327     WHEN NOT MATCHED THEN INSERT *
328     """
329
330     data_upserter=Upsserter(query, "completed_workouts_delta")
331
332     df_start = (spark.readStream
333                 .option("startingVersion", startingVersion)
334                 .option("ignoreDeletes", True)
335                 .option("withEventTimeOrder", "true")
336                 .option("maxFilesPerTrigger", self.maxFilesPerTrigger)
337                 .table(f"{self.catalog}.{self.db_name}.workouts")
338                 .filter("action = 'start'")
339                 .selectExpr("user_id", "workout_id", "session_id", "time as start_time")
340                 .withWatermark("start_time", "30 seconds")
341                 .dropDuplicates(["user_id", "workout_id", "session_id", "start_time"])
342
343
```

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python



File Edit View Run Help Last edit was 18 minutes ago Provide feedback

Run all

Connect

Schedule

Share

```
343
344 df_stop = (spark.readStream
345             .option("startingVersion", startingVersion)
346             .option("ignoreDeletes", True)
347             #.option("withEventTimeOrder", "true")
348             #.option("maxFilesPerTrigger", self.maxFilesPerTrigger)
349             .table(f"{self.catalog}.{self.db_name}.workouts")
350             .filter("action = 'stop'")
351             .selectExpr("user_id", "workout_id", "session_id", "time as end_time")
352             .withWatermark("end_time", "30 seconds")
353             #.dropDuplicates(["user_id", "workout_id", "session_id", "end_time"])
354             )
355
356 # State cleanup - Define a condition to clean the state
357 # - stop must occur within 3 hours of start
358 # - stop < start + 3 hours
359 join_condition = [df_start.user_id == df_stop.user_id, df_start.workout_id==df_stop.workout_id, df_start.
360                  session_id==df_stop.session_id,
361                  df_stop.end_time < df_start.start_time + F.expr('interval 3 hour')]
362
363 df_delta = (df_start.join(df_stop, join_condition)
364             .select(df_start.user_id, df_start.workout_id, df_start.session_id, df_start.start_time,
365                    df_stop.end_time)
366             )
367
368 stream_writer = (df_delta.writeStream
369                  foreachBatch(data => {insert, insert})
```

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python



File Edit View Run Help Last edit was 18 minutes ago Provide feedback

Run all

Connect

Schedule

Share

```
366     stream_writer = (df_delta.writeStream
367                       |.foreachBatch(data_upserter.upsert)
368                       |.outputMode("append")
369                       |.option("checkpointLocation", f"{self.checkpoint_base}/completed_workouts")
370                       |.queryName("completed_workouts_upsert_stream")
371                       )
372
373     spark.sparkContext.setLocalProperty("spark.scheduler.pool", "silver_p1")
374
375     if once == True:
376         return stream_writer.trigger(availableNow=True).start()
377     else:
378         return stream_writer.trigger(processingTime=processing_time).start()
379
380
381
382 def upsert_workout_bpm(self, once=True, processing_time="15 seconds", startingVersion=0):
383     from pyspark.sql import functions as F
384
385     #Idempotent - Only one user workout session completes. So ignore the duplicates and insert the new records
386     query = f"""
387     MERGE INTO {self.catalog}.{self.db_name}.workout_bpm a
388     USING workout_bpm_delta b
389     ON a.user_id=b.user_id AND a.workout_id = b.workout_id AND a.session_id=b.session_id AND a.time=b.time
390     WHEN NOT MATCHED THEN INSERT *
391     """
```

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python



File Edit View Run Help Last edit was 18 minutes ago Provide feedback

Run all

Connect

Schedule

Share

```
381
382 def upsert_workout_bpm(self, once=True, processing_time="15 seconds", startingVersion=0):
383     from pyspark.sql import functions as F
384
385     #Idempotent - Only one user workout session completes. So ignore the duplicates and insert the new records
386     query = f"""
387     MERGE INTO {self.catalog}.{self.db_name}.workout_bpm a
388     USING workout_bpm_delta b
389     ON a.user_id=b.user_id AND a.workout_id = b.workout_id AND a.session_id=b.session_id AND a.time=b.time
390     WHEN NOT MATCHED THEN INSERT *
391     """
392
393     data_upserter=Upsserter(query, "workout_bpm_delta")
394
395     df_users = spark.read.table("users")
396
397     df_completed_workouts = (spark.readStream
398                             .option("startingVersion", startingVersion)
399                             .option("ignoreDeletes", True)
400                             .option("withEventTimeOrder", "true")
401                             .option("maxFilesPerTrigger", self.maxFilesPerTrigger)
402                             .table(f"{self.catalog}.{self.db_name}.completed_workouts")
403                             .join(df_users, "user_id")
404                             .selectExpr("user_id", "device_id", "workout_id", "session_id", "start_time",
405                                         "end_time")
406                             .withWatermark("end_time", "30 seconds"))
```

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python



File Edit View Run Help Last edit was 19 minutes ago Provide feedback

Run all

Connect

Schedule

Share

```
407
408     df_bpm = (spark.readStream
409                 .option("startingVersion", startingVersion)
410                 .option("ignoreDeletes", True)
411                 #.option("withEventTimeOrder", "true")
412                 #.option("maxFilesPerTrigger", self.maxFilesPerTrigger)
413                 .table(f"{self.catalog}.{self.db_name}.heart_rate")
414                 .filter("valid = True")
415                 .selectExpr("device_id", "time", "heartrate")
416                 .withWatermark("time", "30 seconds")
417                 )
418
419     # State cleanup - Define a condition to clean the state
420     # - Workout could be a maximum of three hours
421     # - workout must end within 3 hours of bpm
422     # - workout.end < bpm.time + 3 hours
423     join_condition = [df_completed_workouts.device_id == df_bpm.device_id,
424                      df_bpm.time > df_completed_workouts.start_time, df_bpm.time <= df_completed_workouts.end_time,
425                      df_completed_workouts.end_time < df_bpm.time + F.expr('interval 3 hour')]
426
427     df_delta = (df_bpm.join(df_completed_workouts, join_condition)
428                 .select("user_id", "workout_id", "session_id", "start_time", "end_time", "time", "heartrate")
429                 )
430
431     stream_writer = (df_delta.writeStream
432                      .foreachBatch(data_upserter.upsert)
433                      .outputMode("append")
```

+ New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python



File Edit View Run Help Last edit was 19 minutes ago Provide feedback

▶ Run all

● Connect ▾

📅 Schedule

Share

```
431         stream_writer = (dt_delta.writeStream
432                             .foreachBatch(data_upserter.upsert)
433                             .outputMode("append")
434                             .option("checkpointLocation", f"{self.checkpoint_base}/workout_bpm")
435                             .queryName("workout_bpm_upsert_stream")
436                             )
437
438     spark.sparkContext.setLocalProperty("spark.scheduler.pool", "silver_p2")
439
440     if once == True:
441         return stream_writer.trigger(availableNow=True).start()
442     else:
443         return stream_writer.trigger(processingTime=processing_time).start()
444
445     def _await_queries(self, once):
446         if once:
447             for stream in spark.streams.active:
448                 stream.awaitTermination()
449
450     def upsert(self, once=True, processing_time="5 seconds"):
451         import time
452         start = int(time.time())
453         print(f"\nExecuting silver layer upsert ...")
454         self.upsert_users(once, processing_time)
455         self.upsert_gym_logs(once, processing_time)
456         self.upsert_user_profile(once, processing_time)
457         self.upsert_workouts(once, processing_time)
```

adb-3007342644227763.3.azuredatabricks.net/?o=3007342644227763#notebook/3430256633760883/command/3430256633760889

Microsoft Azure databricks

Search data, notebooks, recents, and more... CTRL + P

sbit-ws ▾ prashant@scholarnest.com ▾

New

- Workspace
- Recents
- Data
- Workflows
- Compute
- SQL
- SQL Editor
- Queries
- Dashboards
- Alerts
- Query History
- SQL Warehouses
- Data Engineering
- Job Runs
- Data Ingestion
- Delta Live Tables
- Machine Learning

05-silver initial-feature Python ☆

File Edit View Run Help Last edit was 19 minutes ago Provide feedback

▶ Run all ● Connect ▾ Schedule Share

```
446         if once:
447             for stream in spark.streams.active:
448                 stream.awaitTermination()
449
450     def upsert(self, once=True, processing_time="5 seconds"):
451         import time
452         start = int(time.time())
453         print(f"\nExecuting silver layer upsert ...")
454         self.upsert_users(once, processing_time)
455         self.upsert_gym_logs(once, processing_time)
456         self.upsert_user_profile(once, processing_time)
457         self.upsert_workouts(once, processing_time)
458         self.upsert_heart_rate(once, processing_time)
459         self._await_queries(once)
460         print(f"Completed silver layer 1 upsert {int(time.time()) - start} seconds")
461         self.upsert_user_bins(once, processing_time)
462         self.upsert_completed_workouts(once, processing_time)
463         self._await_queries(once)
464         print(f"Completed silver layer 2 upsert {int(time.time()) - start} seconds")
465         self.upsert_workout_bpm(once, processing_time)
466         self._await_queries(once)
467         print(f"Completed silver layer 3 upsert {int(time.time()) - start} seconds")
468
469
470     def assert_count(self, table_name, expected_count, filter="true"):
471         print(f"Validating record counts in {table_name}...", end='')
472         actual_count = spark.sql(f"SELECT COUNT(*) FROM {table_name} WHERE {filter}").collect()[0][0]
```

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

initial-feature

Python

☆

File Edit View Run Help

Last edit was 19 minutes ago

Provide feedback

Run all

Connect

Schedule

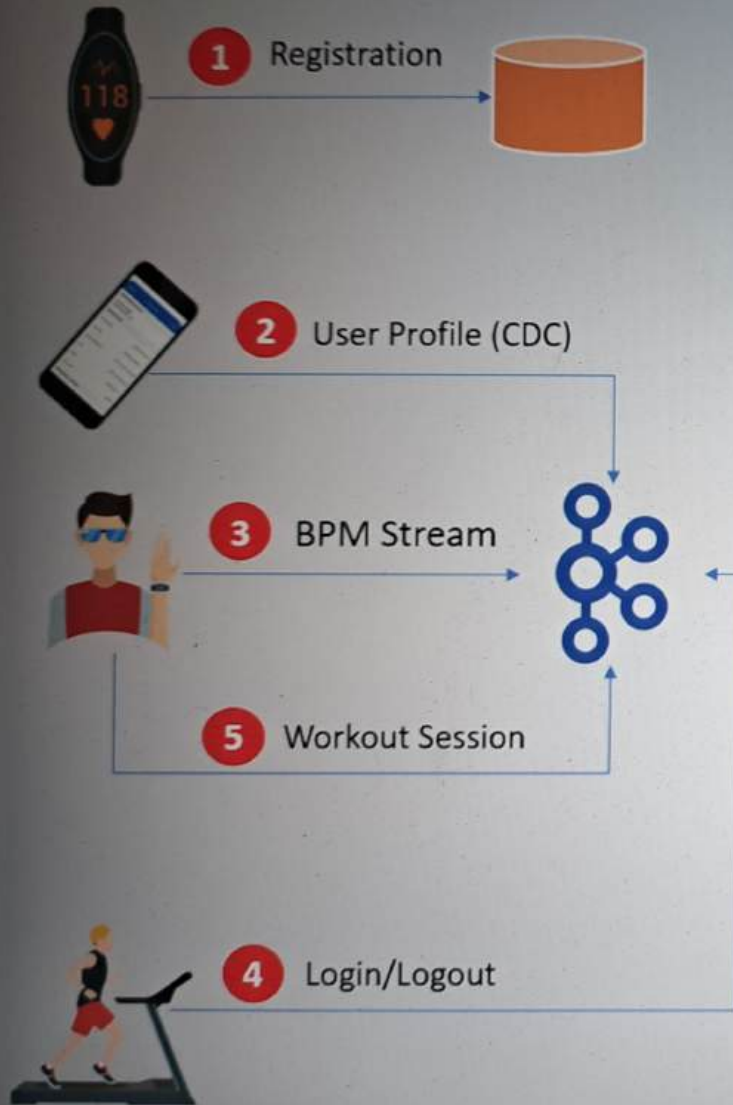
Share

```

469
470 def assert_count(self, table_name, expected_count, filter="true"):
471     print(f"Validating record counts in {table_name}...", end='')
472     actual_count = spark.read.table(f"{self.catalog}.{self.db_name}.{table_name}").where(filter).count()
473     assert actual_count == expected_count, f"Expected {expected_count:,} records, found {actual_count:,} in {table_name} where {filter}"
474     print(f"Found {actual_count:,} / Expected {expected_count:,} records where {filter}: Success")
475
476 def validate(self, sets):
477     import time
478     start = int(time.time())
479     print(f"\nValidating silver layer records...")
480     self.assert_count("users", 5 if sets == 1 else 10)
481     self.assert_count("gym_logs", 8 if sets == 1 else 16)
482     self.assert_count("user_profile", 5 if sets == 1 else 10)
483     self.assert_count("workouts", 16 if sets == 1 else 32)
484     self.assert_count("heart_rate", sets * 253801)
485     self.assert_count("user_bins", 5 if sets == 1 else 10)
486     self.assert_count("completed_workouts", 8 if sets == 1 else 16)
487     self.assert_count("workout_bpm", 3968 if sets == 1 else 8192)
488     print(f"Silver layer validation completed in {int(time.time()) - start} seconds")
    
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text



1. Read new data from the bronze table.
2. Remove duplicates.
3. Transform the raw data.
4. Merge the transformed data to the silver table.

New

Workspace

Recents

Data

Workflows

Compute

SQL

SQL Editor

Queries

Dashboards

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

Delta Live Tables

Machine Learning

05-silver

Initial-feature

Python

File Edit View Run Help

Last edit was 32 minutes ago

Provide feedback

Run all

Connect

Schedule

Share

```
CREATE INTO {self.catalog}.{self.db_name}.users_delta
USING users_delta b
ON a.user_id=b.user_id
WHEN NOT MATCHED THEN INSERT *
```

```
data_upserter=Upsserter(query, "users_delta")
```

```
# Spark Structured Streaming accepts append only sources.
```

```
# - This is not a problem for silver layer streams because bronze layer is insert only
```

```
# - However, you may want to allow bronze layer deletes due to regulatory compliance
```

```
# Spark Structured Streaming throws an exception if any modifications occur on the table being used as a source
```

```
# - This is a problem for silver layer streaming jobs.
```

```
# - ignoreDeletes allows to delete records on partition column in the bronze layer without exception on silver layer streams
```

```
# Starting version is to allow you to restart your stream from a given version just in case you need it
```

```
# - startingVersion is only applied for an empty checkpoint
```

```
# Limiting your input stream size is critical for running on limited capacity
```

```
# - maxFilesPerTrigger/maxBytesPerTrigger can be used with the readStream
```

```
# - Default value is 1000 for maxFilesPerTrigger and maxBytesPerTrigger has no default value
```

```
# - The recommended maxFilesPerTrigger is equal to #executors assuming auto optimize file size to 128 MB
```

```
df_delta = (spark.readStream
```

```
    .option("startingVersion", startingVersion)
```

```
    .option("ignoreDeletes", True)
```

```
    .option("withEventTimeOrder", "true")
```

```
    .option("maxFilesPerTrigger", self.maxFilesPerTrigger)
```

```
    .table(f"{self.catalog}.{self.db_name}.registered_users_bz")
```

